Blockchain-Based Library Book Lending Tracker

Technical Report, Individual Assignment 2 (Formative 2): Transactions, Pending Pool and Mining

Name: Golden Sash Munyankindi

Date: October 2026

GitHub repository: [add your repository link]

Demo video: [add your video link]

1. Introduction

In Formative 1 I built a command line program in C that records every borrow, return and overdue event as a signed block in a SHA-256 hash chain. A block was added to the chain as soon as the librarian recorded the event.

In this assignment I extended the same program so that lending events, token transactions and mining all work together in one flow:

- Returning a book now creates a token reward transaction automatically: 10 coins for an on-time return and 5 coins for a late return. A book that is overdue and never returned creates no transaction.
- New blocks are not added to the chain straight away. They wait in a pending pool until a miner confirms them with proof of work. Only then is the lending record final and the member's balance updated.
- Member balances can be kept with the UTXO model or the account-based model. The model is chosen when the program starts and applies to the whole session.
- Mining can be done solo, in a pool or with rented cloud hash power.

Note on wording: the assignment brief sometimes says "attendance marking" and "absent student". In my library system the equivalent is a lending event (a borrow, return or overdue mark) and a book that is overdue and never returned. I use the library terms in this report.

2. What Changed from Formative 1

The code is now split into 16 source files. I kept the Formative 1 modules (registry, auth, crypto, input) and extended blockchain.c and main.c. The new modules are:

- **transaction.c:** the Transaction struct, its SHA-256 ID and coin amounts.
- **utxo.c:** the UTXO set, coin selection, change outputs, fees and double-spend checks.
- **account.c:** accounts with balance and nonce, and a linked-list history per account.
- **ledger.c:** the log of confirmed transactions, which is sent to whichever model was chosen.
- **pending.c:** the pending pool of unconfirmed blocks.
- **mining.c:** solo, pool and cloud mining, the difficulty setting and a benchmark.

- **lending.c, tokens.c, app.c:** the menu actions and the shared program state.
- **storage.c:** safe saving and loading of binary files, used for the chain, the pool and the ledger.

2.1 Updated block structure

All Formative 1 fields are unchanged. I added the two fields the brief asks for, plus three fields that proof of work needs:

Table 1: Block fields

Field	Type	Purpose
index, timestamp, book_id, book_title, member_id, member_name, action, recorded_by, previous_hash, signature, sig_len, hash	(as in Formative 1)	Unchanged
token_reward	int	NEW: coins earned. 10 for an on-time return, 5 for a late return, 0 otherwise
tx_id	char[65]	NEW: SHA-256 hash of the reward (or transfer) transaction, or 64 zeros if the block has no transaction
difficulty	int	Number of leading zeros this block was mined to (1 to 4)
mined_by	char[40]	Who confirmed the block: a username, POOL or CLOUD:<user>
nonce	unsigned long long	Incremented by the miner until the hash meets the difficulty

Each block stores its own difficulty, so if the difficulty changes later, older blocks can still be checked against the target they were mined at. The action field now also allows TRANSFER, for manual token transfers between members.

3. How Lending Events Trigger Token Transactions

Each lending action creates a new block, and the action decides whether a transaction is created:

Table 2: Lending events and transactions

Event	Token reward	Transaction created
BORROWED	0	None
OVERDUE (book not returned)	0	None. The block records that the book is missing, but no coins move
RETURNED on time	10	REWARD: LIBRARY pays 10 coins: 9 to the member and a 1-coin fee to the miner

RETURNED late	5	REWARD: LIBRARY pays 5 coins: 4 to the member and a 1-coin fee to the miner
TRANSFER (manual)	0	TRANSFER: member to member, the sender also pays the 1-coin fee

A return is late if more than 14 days have passed since the borrow, or if a librarian has already marked the loan OVERDUE. The second rule exists because a demonstration cannot wait 14 days.

When a book is returned, lending_return() in lending.c does the following:

1. It finds the open loan by looking at the confirmed chain and then the pending pool, which together form one timeline. This way a book that is borrowed in a pending block already counts as on loan.
2. It decides whether the return is on time or late, and sets the reward to 10 or 5.
3. It builds the reward transaction: sender LIBRARY, recipient the member, amount = reward - 1, fee = 1. The transaction ID is the SHA-256 hash of its fields.
4. It asks the ledger to admit the transaction. The UTXO model chooses and locks the library's coins; the account model checks the library's nonce and balance.
5. It creates the RETURNED block with token_reward and tx_id filled in, signs it with the librarian's ECDSA private key, and puts it in the pending pool together with the transaction.

The reward and its transaction ID are part of the signed content of the block, so the block and its transaction are bound together. Anyone who changes either one breaks the signature or the link.

Where the coins come from: when the genesis block is created, a coinbase transaction gives the LIBRARY treasury 10,000 coins, and the genesis block's tx_id points to it. Members start with a balance of 0 and can only earn coins from confirmed returns (or receive them in a transfer from another member).

4. The Pending Pool and How Mining Confirms Blocks

The pending pool (pending.c) is a first-in, first-out queue of PendingEntry structs. Each entry holds a signed block and, for returns and transfers, its transaction. It is saved to pending.dat after every change, so it survives a restart. A pending block has no index, previous_hash or nonce yet, because its position in the chain is only decided when it is mined.

While a block is pending, its transaction has already passed the model's checks and the sender's funds are reserved, but no balance has changed. Mining is the confirmation step. For each block, oldest first, the miner:

6. Validates the block: the librarian's signature must verify with their public key, the attached transaction must hash to the block's tx_id, and its amount must match token_reward. An invalid block is dropped and its reserved funds are released.
7. Places the block in the chain: it sets index = chain length, previous_hash = hash of the last block, and records the difficulty and the miner's name.
8. Does proof of work: starting from nonce 0, it hashes the block and adds 1 to the nonce until the SHA-256 hash starts with the required number of zeros.

9. Appends the block to the chain and confirms its transaction in the ledger. If the ledger rejects the transaction, the block is removed again, so the chain and the ledger never disagree.
10. Saves the chain, the ledger and the pool, and prints the UTXO set (UTXO model) or the updated accounts (account model).

After the pool has been mined, the miner is paid with a new coinbase transaction: 50 coins for each confirmed block plus the fees of the transactions inside them.

When the program starts again, the pending transactions are checked again under the model chosen for the new session. This is similar to a node rebuilding its memory pool. Anything that is no longer valid is dropped with a warning.

5. Transaction Models

Both models work with the same Transaction struct. Its body holds the type, sender, recipient, amount, fee, nonce, timestamp and a short memo, and the transaction ID is the SHA-256 hash of the body. Amounts are stored as whole numbers of hundredths of a coin (the same idea as satoshis), so fees and pool shares can be divided exactly, without floating-point rounding errors.

Every confirmed transaction is appended, in order, to a log in ledger.c, which is saved as ledger.dat. At startup the log is replayed into the chosen model. Because both models are built from the same log, switching between them does not change any balance; only the way the balances are stored and checked changes. Figures 8 and 13 show the same balances under each model.

5.1 UTXO model (utxo.c)

The UTXO set is a dynamic array of Utxo structs, each with txid, vout, owner, amount and two status fields. A balance is not stored anywhere: it is the sum of the outputs a member owns.

- **Inputs and outputs:** a transaction spends whole earlier outputs as inputs. The program chooses the sender's oldest free outputs until they cover amount + fee. Output 0 pays the recipient.
- **Fee:** the fee is not an output. It is the difference between the inputs and the outputs, and it goes to the miner in the coinbase transaction. Every reward and transfer has a fixed 1-coin fee.
- **Change:** if the inputs are worth more than amount + fee, output 1 sends the excess back to the sender as change. For example, John spends his 9-coin output to send 3 coins: 3 go to the recipient, 5 come back as change and 1 is the fee (Figure 10).
- **Insufficient inputs:** if the sender's free outputs cannot cover amount + fee, the transaction is rejected.
- **Double-spend prevention:** when a transaction enters the pending pool, its inputs are marked reserved_by = txid, so no other pending transaction can use them. When the transaction is confirmed, its inputs are removed from the set. A transaction that names an input that is missing (already spent), reserved by another transaction, owned by someone else or listed twice is rejected (Figure 12).
- **Validation at confirmation:** all inputs are checked again, and they must all be confirmed. The set is only changed after every check has passed.
- **Display:** the full UTXO set, with each output's owner and status, is printed after every confirmed block.

A design detail I had to add: the library treasury starts as a single 10,000-coin output. After the first return, that output was locked by the pending reward, so a second return was rejected until the first one had been mined. Real Bitcoin nodes let a sender spend the change of their own unconfirmed transaction, so I did the same. When a transaction is admitted, its change output is added to the set as "unconfirmed change" that only the sender may spend. Payments to other people only appear after mining, so a member still cannot spend a reward before it is confirmed. Figure 7 shows the chain of LIBRARY change outputs being confirmed one after another.

5.2 Account-based model (account.c)

Each ledger ID (the registry member IDs, LIBRARY and the miner accounts) has an Account struct with balance, nonce, reserved, pending_count and a pointer to its history list. Every member from members.txt is created with balance 0 at startup.

- **Debit and credit:** confirming a transaction debits the sender amount + fee, credits the recipient the amount and increments the sender's nonce. The fee goes to the miner's coinbase.
- **Nonce:** the nonce counts the sender's outgoing transactions. A new transaction must use exactly the next nonce (confirmed nonce + transactions already waiting in the pool). A nonce that was already used is rejected as a replay, and a nonce that skips ahead is rejected because it would leave a gap (Figure 14). For manual transfers the user types the nonce, so the rejection can be demonstrated. For reward transactions the program uses LIBRARY's next nonce automatically.
- **Insufficient balance:** a transfer is rejected if balance - reserved is less than amount + fee. "Reserved" is the cost of the sender's transactions still waiting in the pool, so the same coins cannot be promised twice.
- **History as a linked list:** every account has a singly linked list of TxHistoryNode structs (head and tail pointers, so adding to the end is O(1)). Each node records the transaction ID, sender, recipient, amount, fee, nonce and time. Menu option 12 walks the list and prints the full history of any account (Figure 16). The nodes are allocated before anything is changed, so if malloc fails no half-finished transaction is left behind, and all lists are freed when the program exits.

5.3 Choosing the model

The model is chosen with the --model utxo|account command line option. If the option is left out, a menu asks at startup. The choice applies to every transaction in the session. Inside ledger.c, every operation (admit, release, confirm, balance) is sent to utxo.c or account.c depending on the chosen model, so the rest of the program does not need to know which model is in use.

5.4 Comparison

Table 3: UTXO and account models compared

	UTXO model	**Account model**
Where the balance lives	Not stored: the sum of unspent outputs	Stored in each account
Stopping replays and double spends	Spent outputs no longer exist; pending inputs are locked	The nonce must be exactly the next one; a used nonce can never be accepted again
Fee	Inputs minus outputs	Explicit field, debited with the

		amount
Change	Extra change output back to the sender	Not needed: only the balance changes
Transaction ordering	Only through which outputs are spent	Strict per-sender order by nonce
History	Rebuilt from the confirmed log	Linked list kept per account
Strengths	Independent outputs can be checked in parallel; better privacy; easy to audit	Simple to understand; small state; easy to show balances
Weaknesses	Coin selection, many small outputs, change handling	Every transaction depends on the sender's current state; the nonce must be managed

In my tests both models accepted and rejected the same transactions for the same reasons, but their error messages show the different mechanisms. A replayed transfer is reported as "input already spent" by the UTXO model and as "nonce already used" by the account model (Figures 12 and 15).

6. Mining Simulation Methodology

6.1 Proof of work

The block hash is SHA-256 over a header made of the index, previous_hash, the hash of the signed content (which includes token_reward and tx_id), the difficulty, mined_by, the signature bytes and finally the nonce. A block is valid when the hexadecimal hash starts with `difficulty` zeros. The difficulty can be 1 to 4 and defaults to 2. It is set with --difficulty or with menu option 8, and any other value is rejected.

Only the nonce changes between attempts, so the miner feeds the fixed part of the header to SHA-256 once and then, for each attempt, copies that saved state and hashes only the nonce digits (the same "midstate" idea Bitcoin miners use). The result is exactly the same as hashing the whole header. The leading-zero test is also done on the raw digest, without converting it to hex. Each hexadecimal digit is 4 bits, so the chance that a hash meets difficulty d is 1/16^d, and the expected number of attempts is 16^d. I measured this with the benchmark (menu option 9), which mines 20 sample blocks at each difficulty:

Table 4: Difficulty benchmark (20 blocks per level)

Difficulty	Target	Expected attempts	Average measured	Min	Max	Avg time
1	0…	16	15	2	68	0.02 ms
2	00…	256	250	9	701	0.16 ms
3	000…	4,096	4,497	116	15,265	1.97 ms
4	0000…	65,536	79,979	415	491,440	24.1 ms

Each extra zero multiplies the expected work by 16. Single blocks vary a lot (min and max), because finding a valid hash is random. In one of my cloud runs a difficulty 2 block needed 1,606 attempts instead of about 256, which shows the same randomness (Figure 24).

6.2 Solo mining

The logged-in user mines every pending block alone. For each block the program shows the nonce, the number of hash attempts, the time and the hash. At the end it shows the total and average attempts compared with the expected 16^d. The miner keeps the full reward: 50 coins per block plus all transaction fees, paid by a coinbase transaction to MINER_<username>.

6.3 Pool mining and the contribution metric

The user chooses 2 to 8 miners. Each one gets a random hash rate between 100 and 500 attempts per round (it can be reproduced with --seed). The miners work at the same time on separate nonces: miner k of n tries k, k+n, k+2n, and so on, so no hash is computed twice. In each round, miner k makes hash_rate attempts spread evenly through the round. The program processes all miners' attempts in time order and stops everyone at the first valid hash. As a result, each miner's number of attempts is proportional to its hash rate, which is how real pools measure work (with "shares").

My first version ran each miner's slice one after another and stopped a miner when it found its own valid hash. With difficulty 2 several miners found one in the same round, so the attempt counts no longer followed the hash rates (the fastest miner got under 10%). Processing the attempts in time order fixed this.

Rewards: the total reward (50 per block plus fees) first has the 2% pool fee taken off for POOL_OPERATOR. The rest is shared using share = miner_attempts / total_attempts x (total reward - pool fee), in whole cents. Any rounding remainder goes to the operator, so every cent is accounted for. The program prints a table with miner ID, hash rate, attempts, share %, blocks found and reward (Figure 11).

6.4 Cloud mining

The user rents hash power for 1 to 5 rounds. The plan gives 400 attempts per round, costs a 20-coin rental fee per round, and has a maintenance fee of 10% of each round's mining reward. In each round the rented power mines pending blocks. If a block is not solved by the end of a round, the search continues from the same nonce in the next round. If the pool is empty, the rented power sits idle but the rental fee is still charged.

Each round's reward minus the maintenance fee is paid to the renter by coinbase, and the maintenance fee goes to CLOUD_HOST. I assumed the rental fee is paid to the provider outside the chain. After every round the program prints the cumulative rewards, fees and net. If the cumulative fees are greater than the cumulative rewards, it prints a warning that the rental is unprofitable. At the end a summary table shows every round, then the gross earnings, total fees paid and net profit (Figures 17 and 18).

Whether a rental makes a profit depends on the difficulty: 400 attempts per round is about 1.6 blocks per round at difficulty 2, but only about 0.1 at difficulty 3. Renting more rounds than there are pending blocks also loses money.

7. Security: SHA-256, Signatures and Immutability

- **SHA-256 everywhere:** for block hashes, the signed content hash, transaction IDs and proof of work. Every block's hash commits to its token_reward and tx_id, and every tx_id commits to the transaction's sender, recipient, amount, fee and nonce.
- **Digital signatures:** as in Formative 1, every block is signed with the recording librarian's ECDSA P-256 private key, which is AES-encrypted and only unlocked at login. It is verified with their public key. The signature covers the reward and the transaction ID, so a miner or an attacker cannot change the reward without the librarian's key.
- **Full validation:** option 15, startup and every action that adds or mines a block check four things for each block: (a) the stored hash equals the recomputed hash, (b) previous_hash links to the block before, (c) the signature verifies, and (d) the hash meets the block's difficulty. They also check that each block's tx_id exists in the ledger, that the transaction's hash is correct and that its amount matches the token reward. Every problem is listed, not only the first.
- **Ledger integrity:** every transaction in ledger.dat is re-hashed when it is loaded. If someone edits a transaction, its ID no longer matches and the program refuses to start (Figure 23).
- **Proof of work and immutability:** to change an old block an attacker must redo its proof of work, and the block's new hash then breaks the next block's previous_hash, so every later block would also have to be re-mined. Figure 21 shows this attack: the late reward is changed from 5 to 10 and the block is re-mined in 323 attempts. Validation still reports the invalid signature, the reward that no longer matches its transaction, and the broken link at the next block. While the chain is invalid, new blocks and mining are refused, so the change is never saved.
- **Access control:** only ADMIN users can change the difficulty, run the tamper demo or add users.

Limitations: this is a single-machine simulation. An attacker who controls the folder and a librarian's password could rebuild the whole chain. A real network stays safe because many nodes would reject a chain with less total work. The mining payouts are protected by their transaction IDs, but they are not referenced by a block, and members do not sign their own transfers (the librarian records them). These would be the next steps in a real system.

8. Screenshots

All screenshots below come from real runs of the program. They were recorded through a terminal session (passwords are hidden), and the repeated main menu has been removed from them to save space.

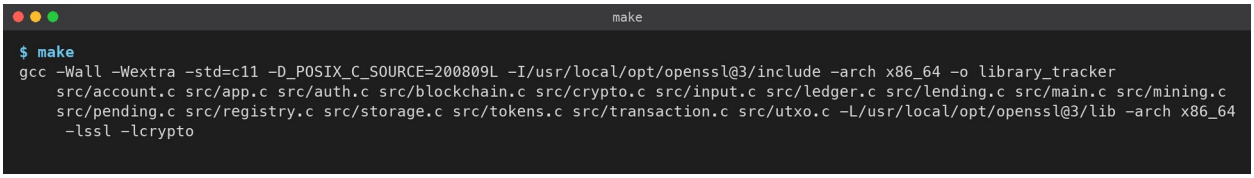


Figure 1: Compiling with make (no warnings with -Wall -Wextra)

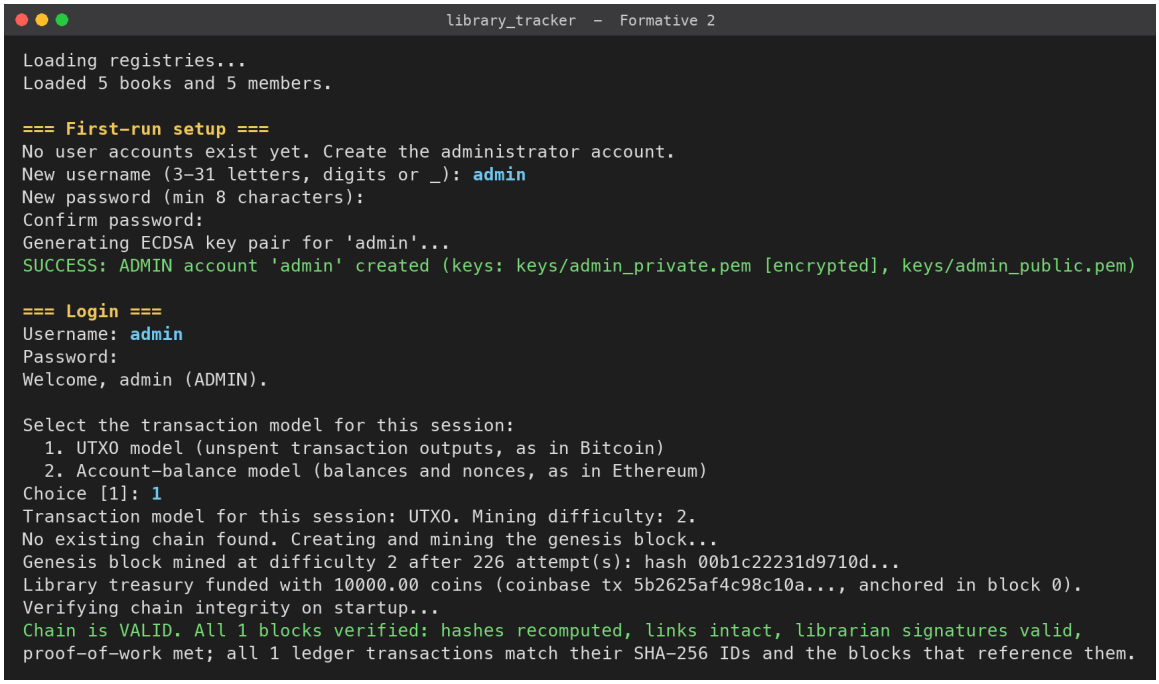


Figure 2: First run: admin account, model selection, genesis block mined and LIBRARY funded

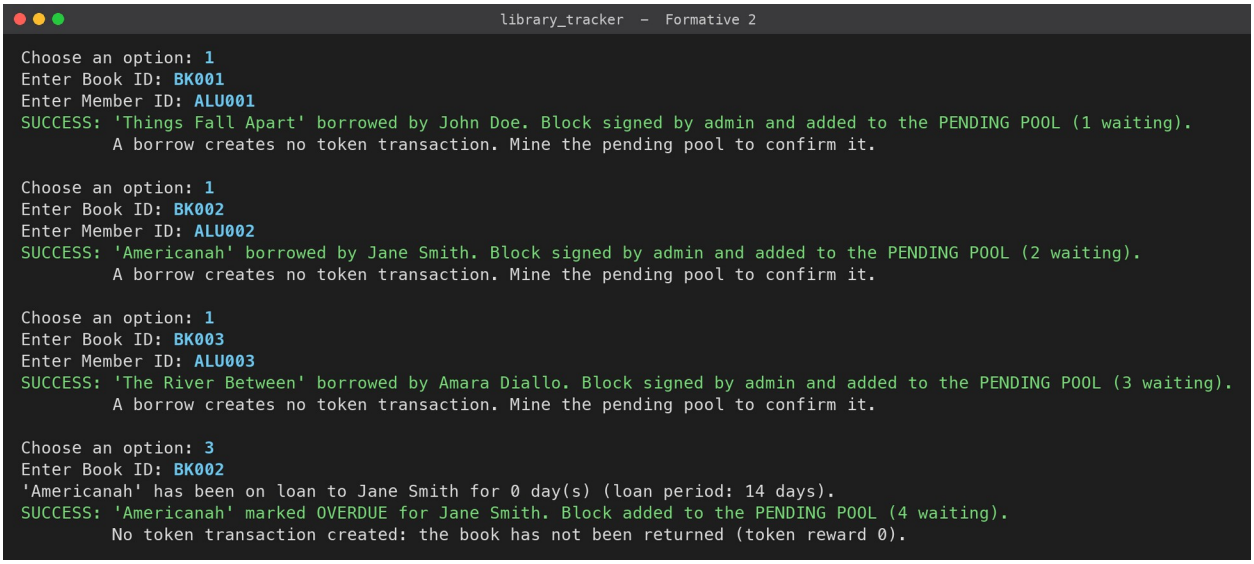


Figure 3: Borrows go to the pending pool. Marking a book overdue creates no transaction

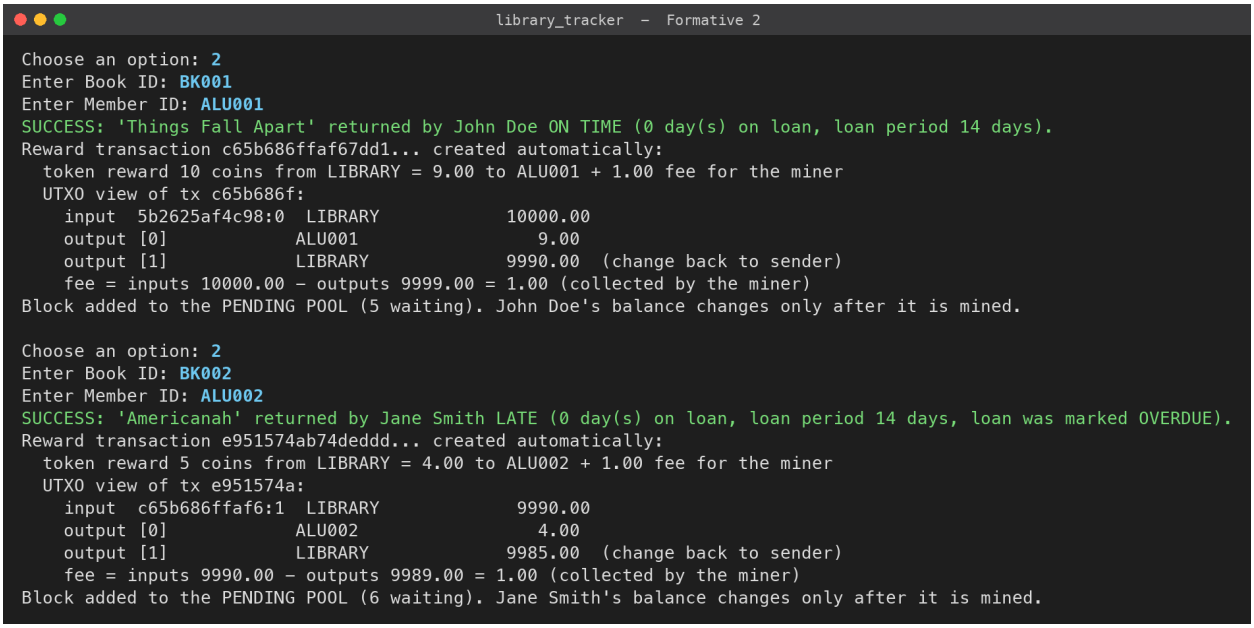


Figure 4: Returns create reward transactions automatically: on time 10 coins, late 5 coins (UTXO view)

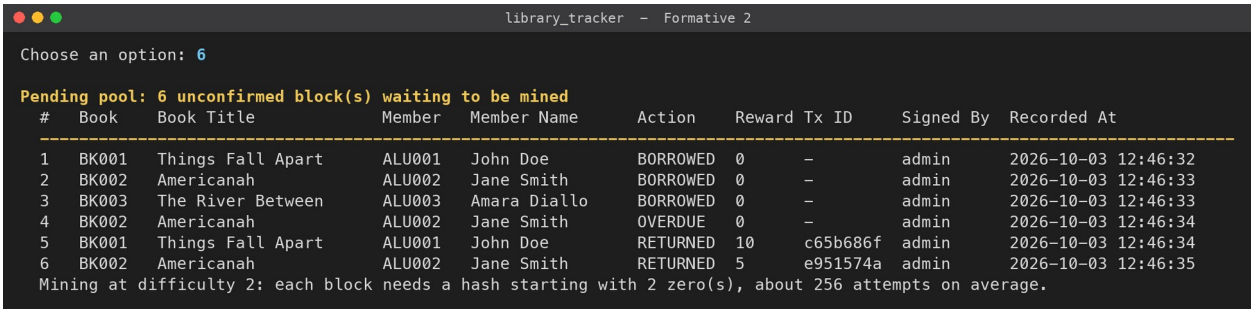


Figure 5: Pending pool with six unconfirmed blocks

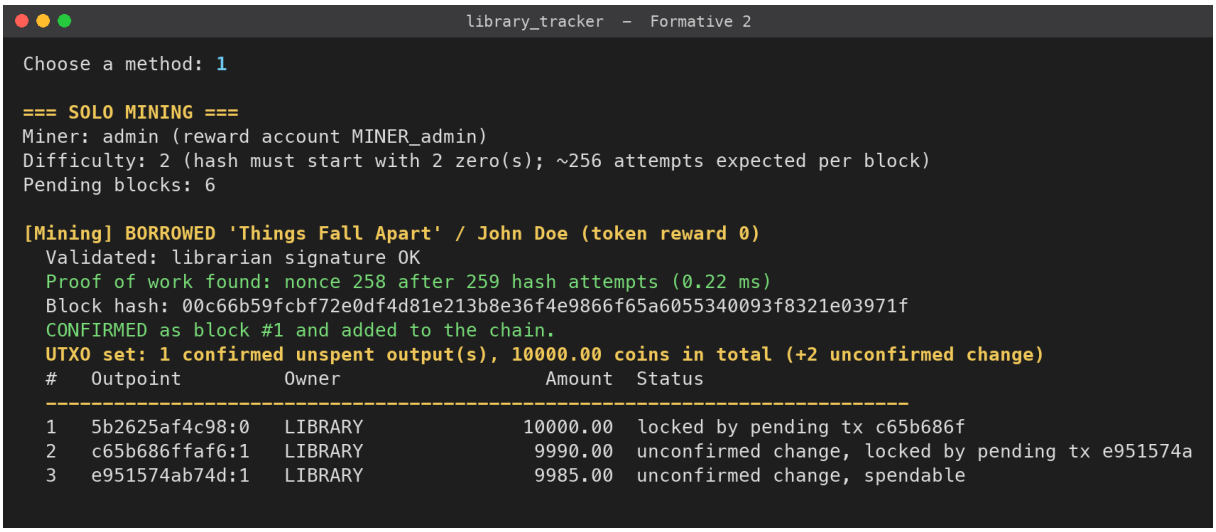


Figure 6: Solo mining: validation, nonce search, hash attempts and the UTXO set after the block

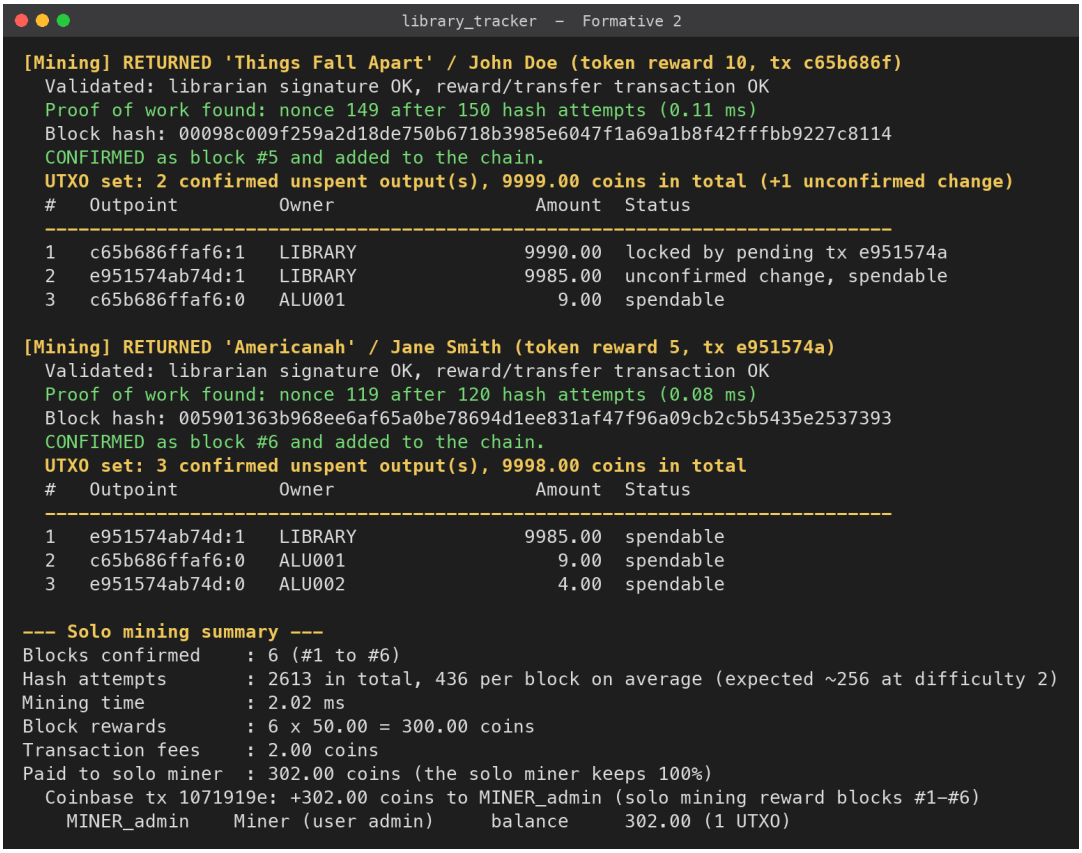


Figure 7: Reward blocks confirmed, members credited, and the solo mining summary with the miner's reward

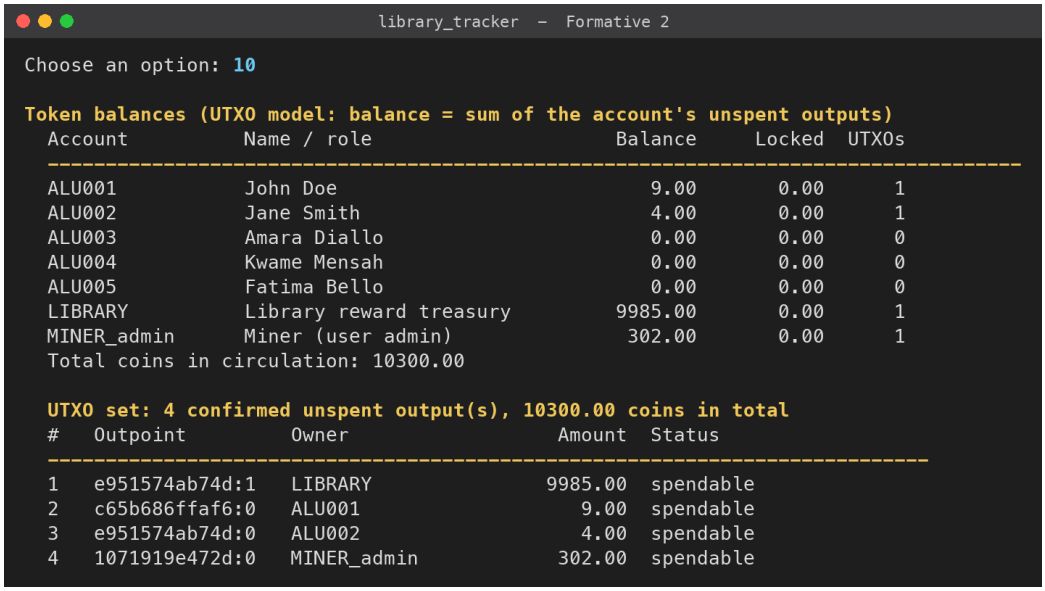


Figure 8: Balances and the full UTXO set after mining (UTXO model)

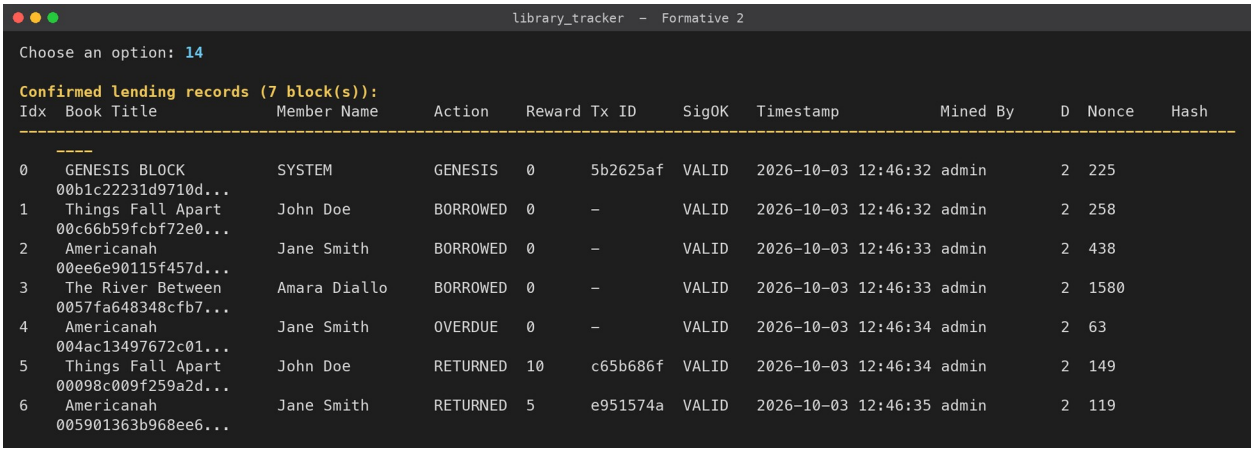


Figure 9: Confirmed lending records with token reward, transaction ID, signature validity, miner, difficulty and nonce

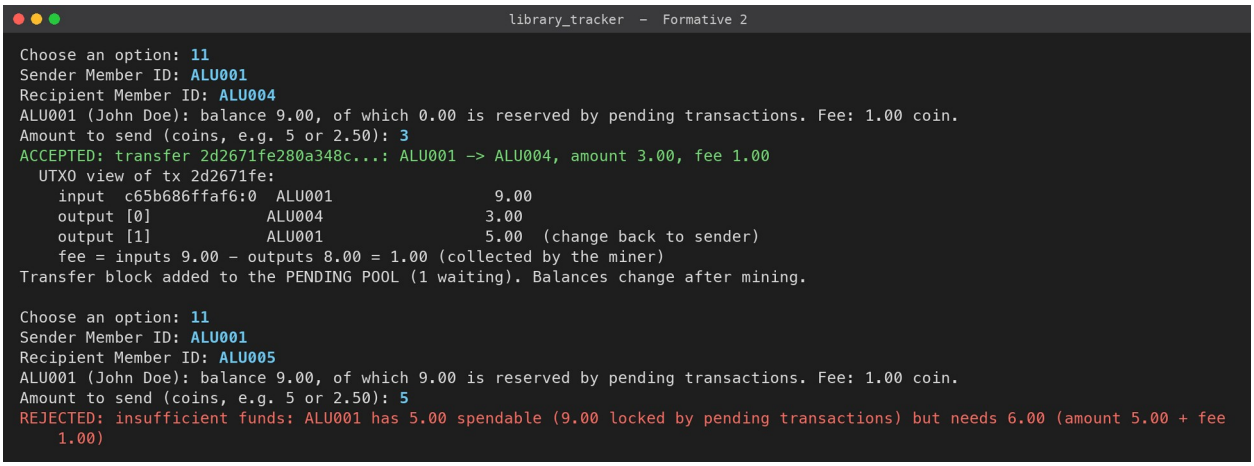


Figure 10: UTXO transfer with a change output, then a transfer rejected because the inputs are locked

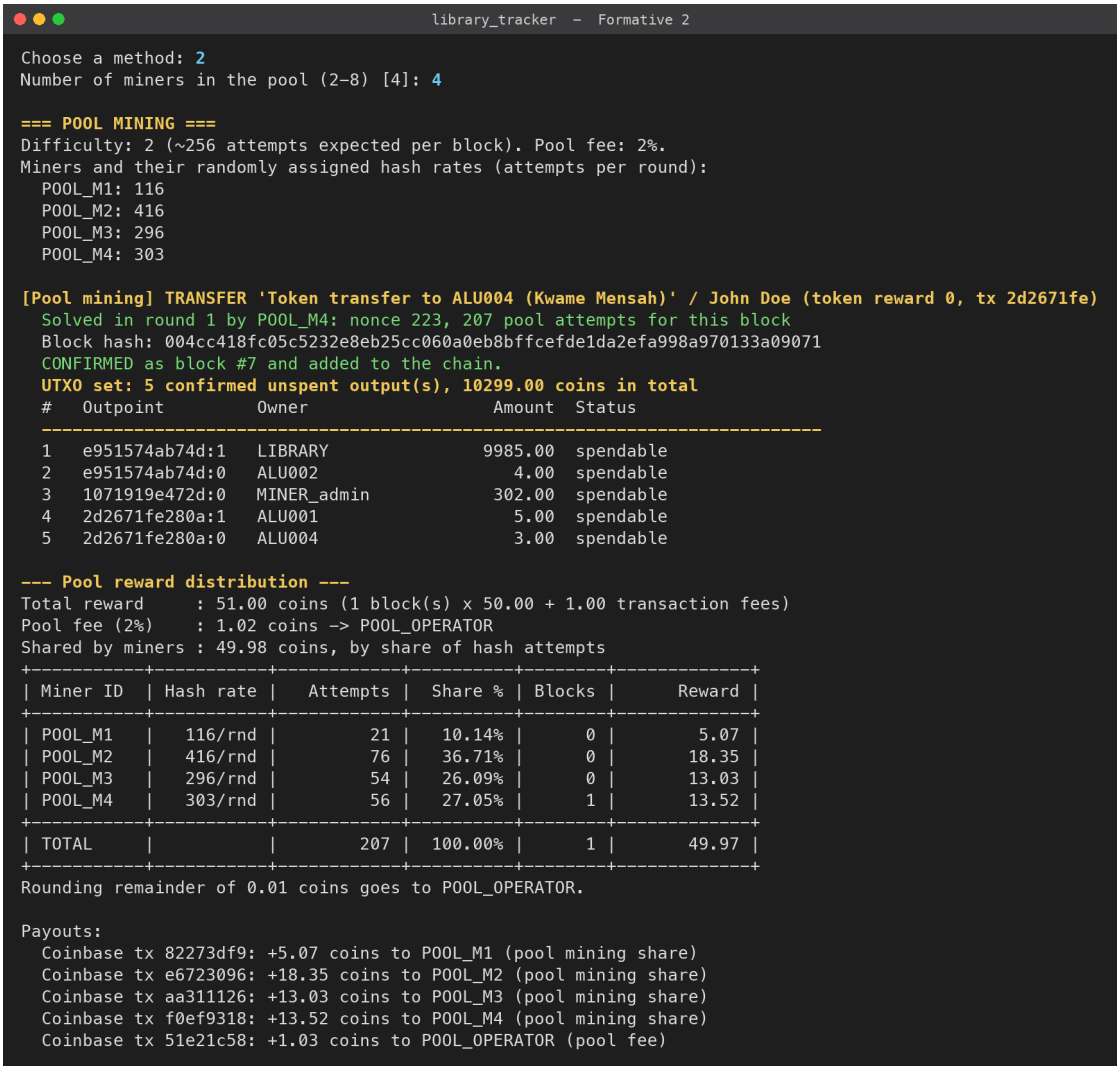


Figure 11: Pool mining: random hash rates, 2% pool fee and reward-sharing table

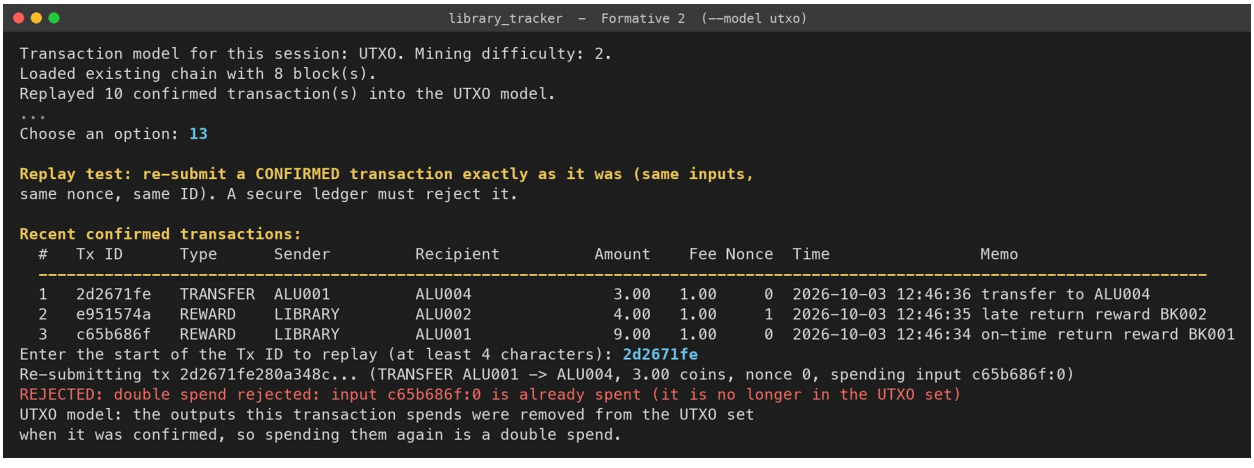


Figure 12: Double spend: replaying a confirmed transfer is rejected (UTXO model)

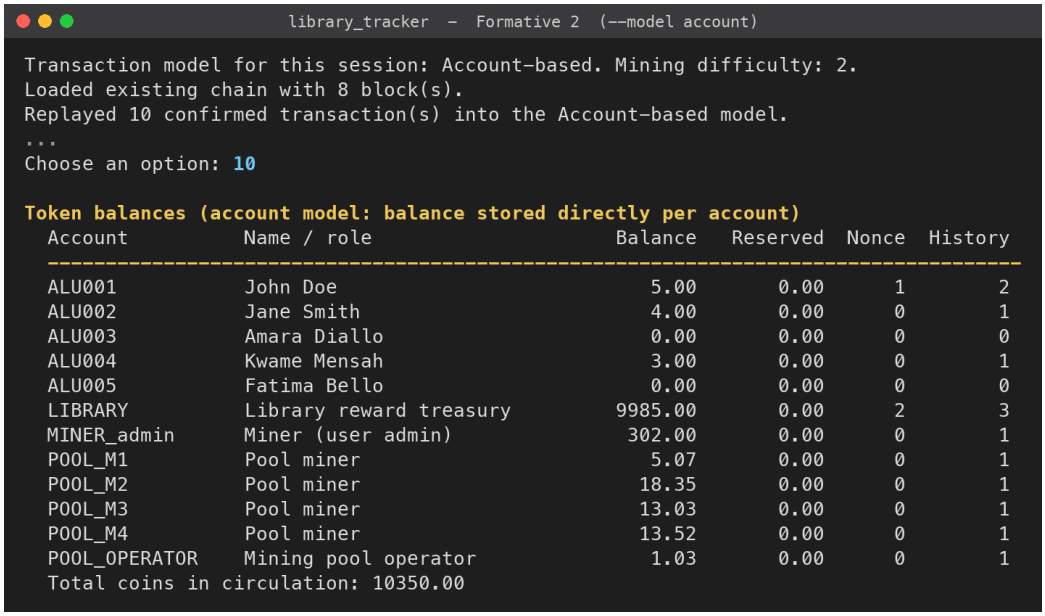


Figure 13: The same log replayed into the account model gives the same balances

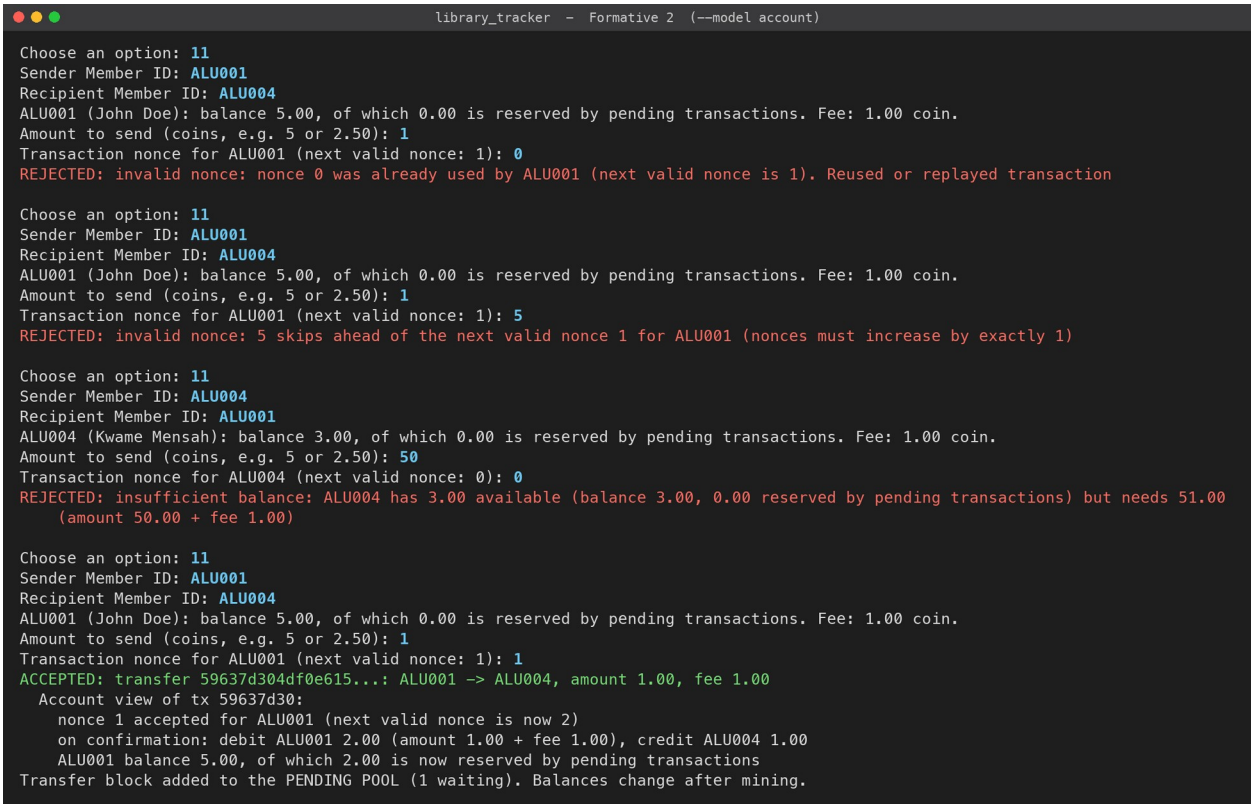


Figure 14: Account model: reused nonce, skipped nonce and insufficient balance rejected; correct nonce accepted

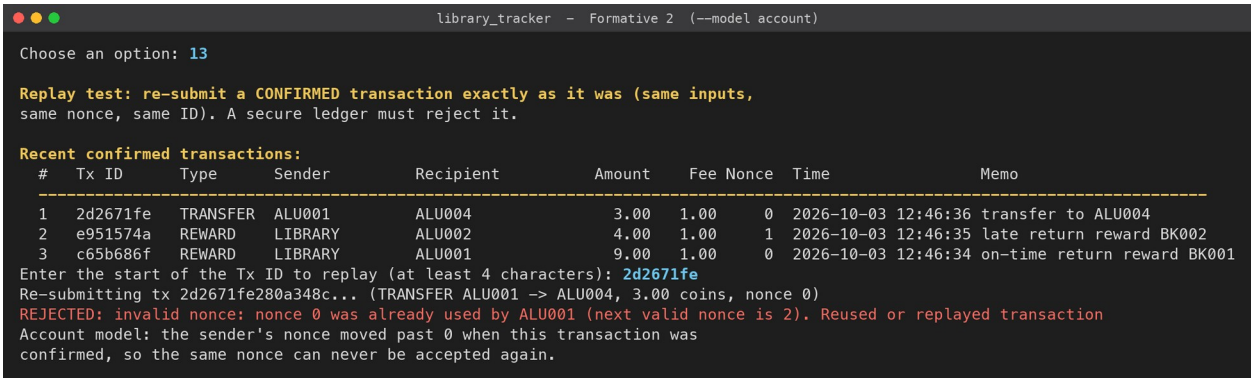


Figure 15: Replay rejected by the account model (nonce already used)

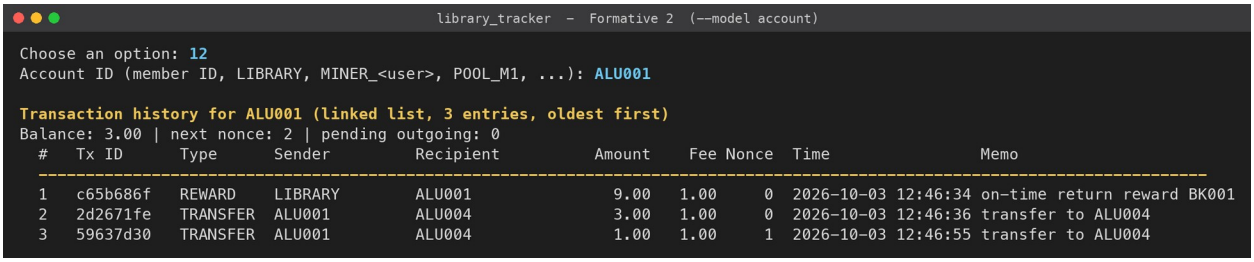


Figure 16: Transaction history printed from the account's linked list

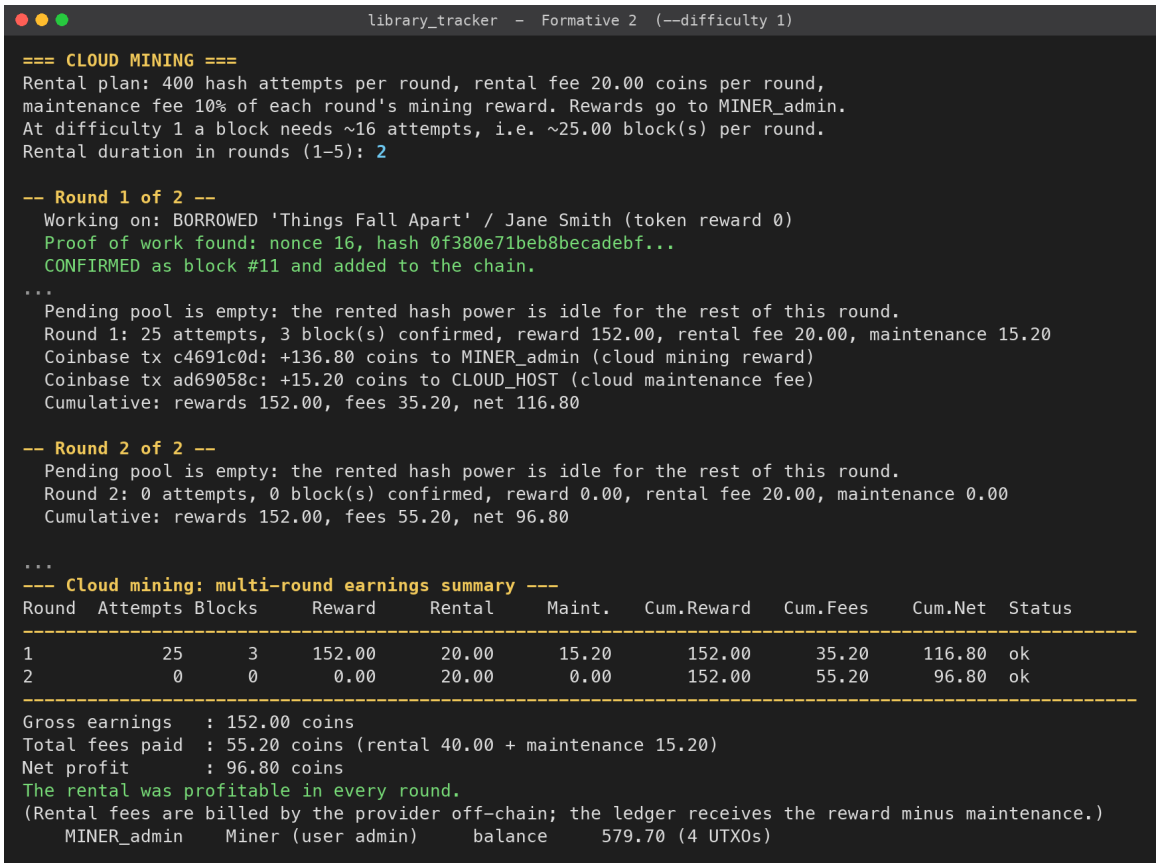


Figure 17: Cloud mining at difficulty 1: profitable rental with a multi-round summary

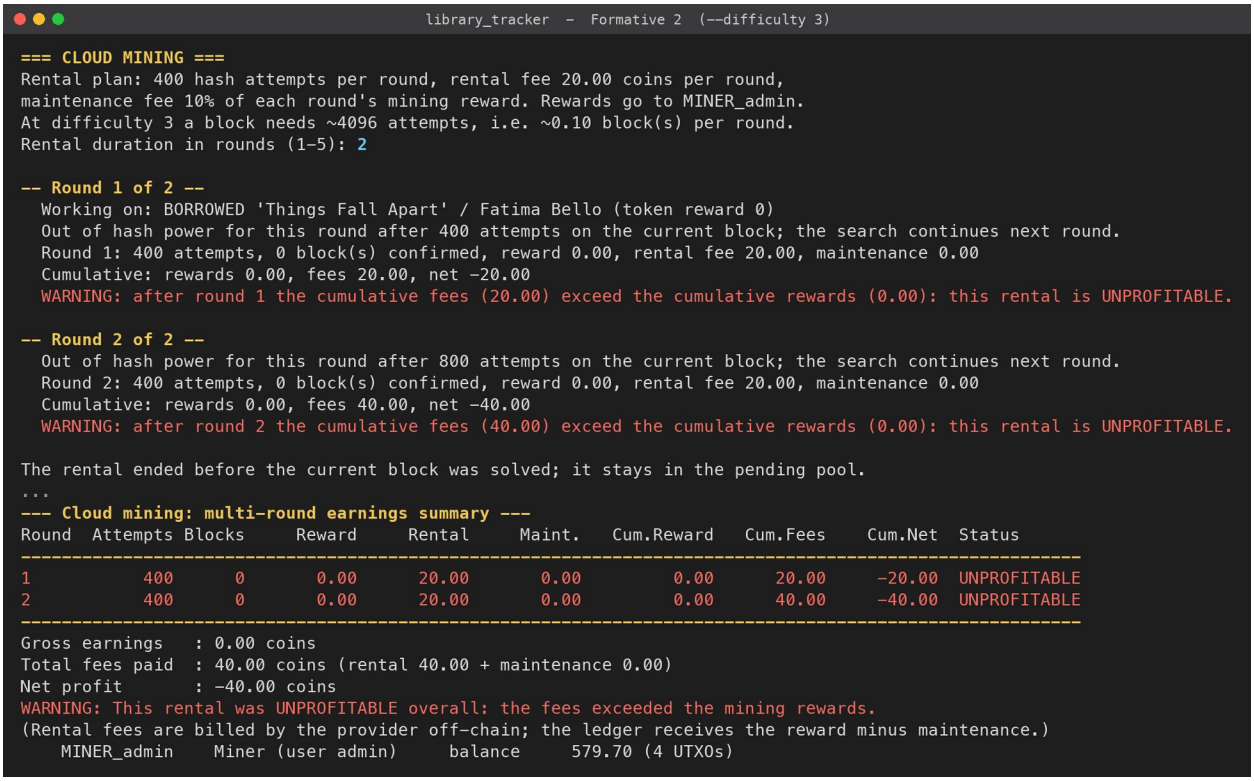


Figure 18: Cloud mining at difficulty 3: unprofitable rental warnings

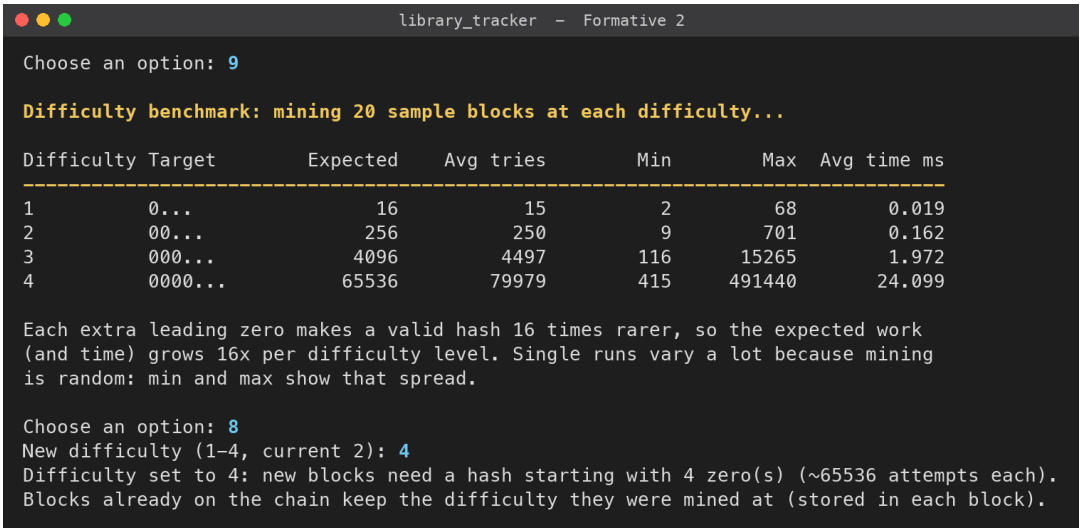


Figure 19: Difficulty benchmark, then the difficulty set to 4 from the menu

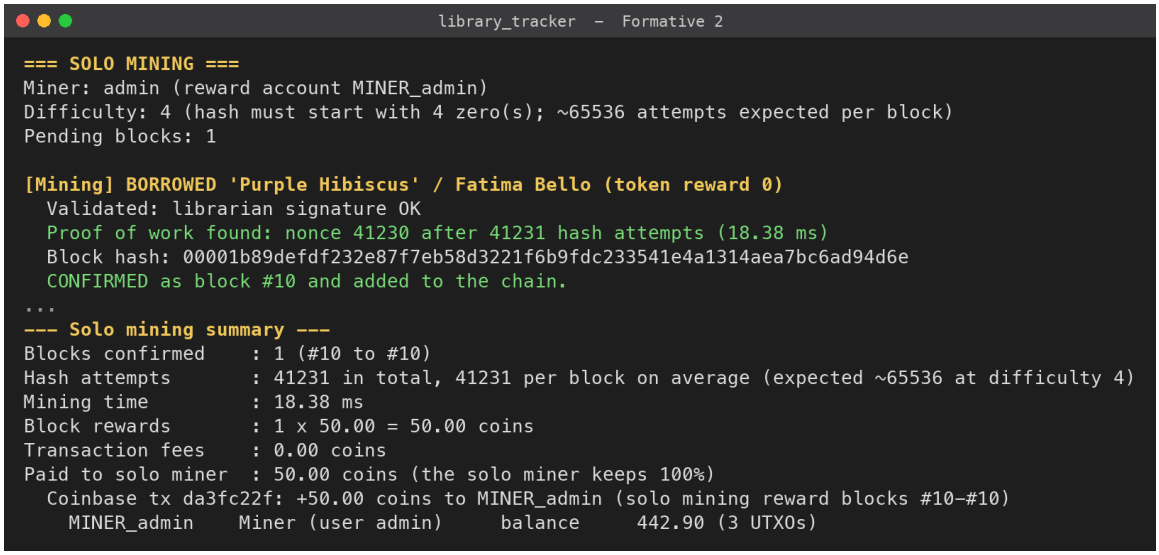


Figure 20: Solo mining at difficulty 4: 41,231 attempts, and the hash starts with 0000

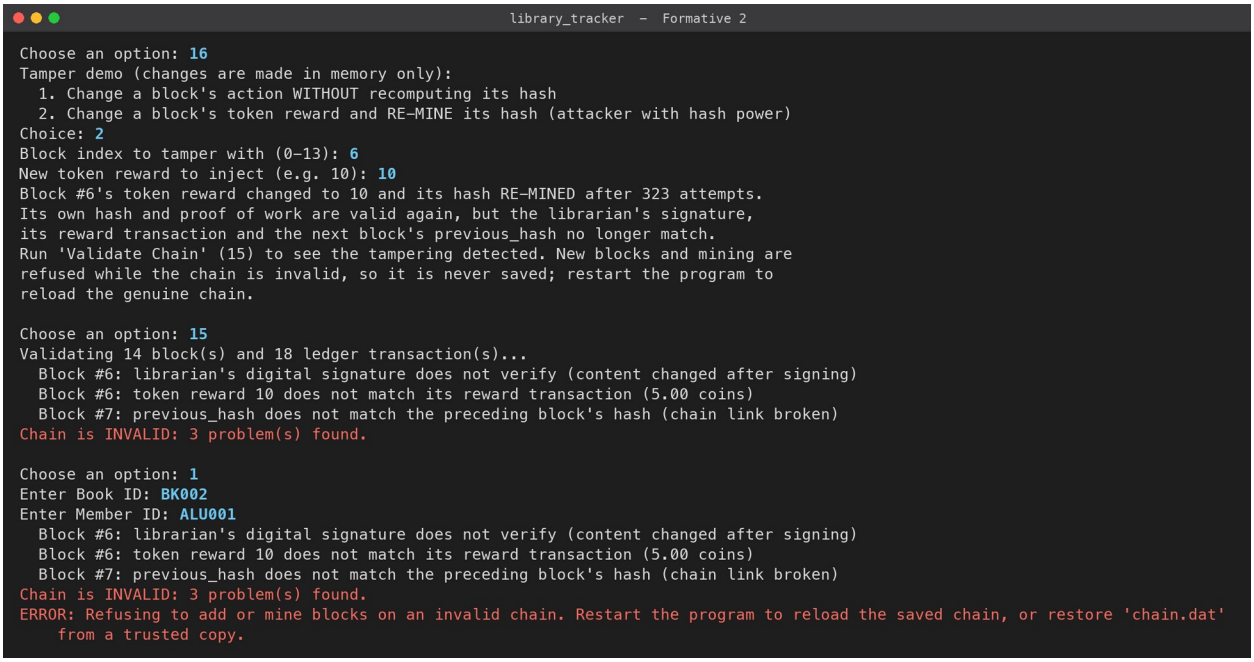


Figure 21: Attack: the reward is changed and the block re-mined. Validation still finds three problems, and new blocks are refused

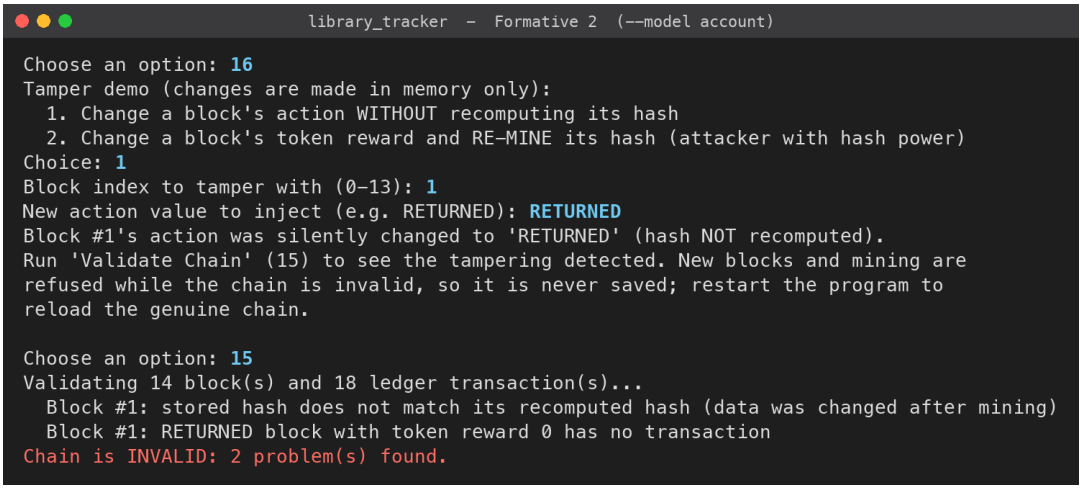


Figure 22: Changing an action without re-hashing is detected

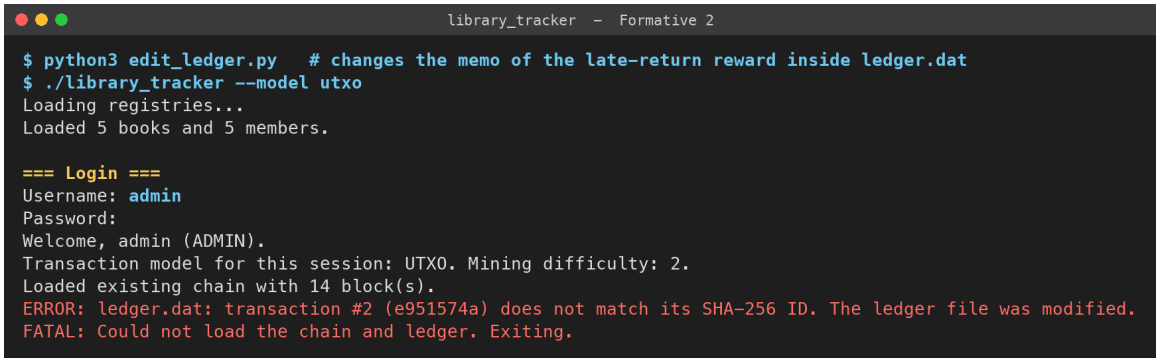


Figure 23: Editing a transaction in ledger.dat is detected at startup

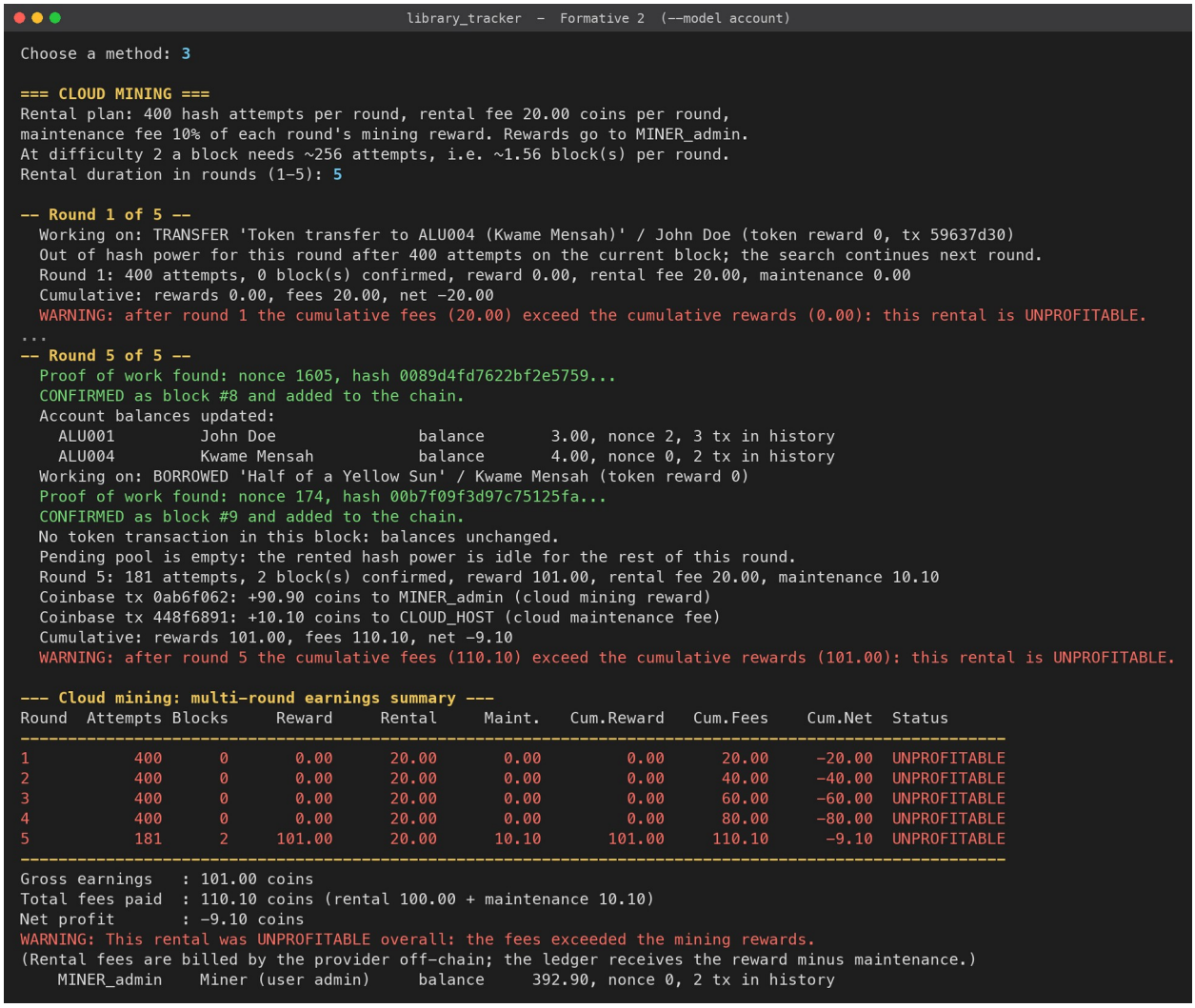


Figure 24: A run where an unlucky block needed 1,606 attempts at difficulty 2 (mining randomness)

9. Test Results for Edge Cases

I tested the program in three ways:

- **Automated tests (make test):** tests/run_tests.sh runs the real program through every scenario in the brief, in a temporary folder, by feeding menu input and checking the output. Its 51 checks cover rewards, the pending pool, the three mining methods, both models, double spends, nonces, difficulty, tamper detection and access control. All 51 pass, and the suite takes about 3 seconds.
- **Manual runs:** the scenarios in the screenshots, plus switching models between runs while transactions are still pending.
- **Tools:** the program compiles with no warnings under -Wall -Wextra and passes the Clang static analyzer. It ran clean under AddressSanitizer and UndefinedBehaviorSanitizer, and the macOS leaks tool reported "0 leaks for 0 total leaked bytes".

The table lists the edge cases and their results.

Table 5: Edge-case test results

#	Test case	Model	Expected	Result
1	Book marked OVERDUE and not returned	Both	Block with reward 0, no transaction, no balance change	Pass (Fig. 3, 9)
2	On-time return	Both	Reward 10: member gets 9, miner gets the 1 fee after mining	Pass (Fig. 4, 7)
3	Late return (loan marked overdue)	Both	Reward 5: member gets 4	Pass (Fig. 4, 7)
4	Balance before mining	Both	Unchanged until the block is mined	Pass (Fig. 4, 7)
5	Transfer larger than the balance	Account	Rejected: insufficient balance	Pass (Fig. 14)
6	Transfer while the inputs are locked by a pending transfer	UTXO	Rejected: insufficient funds (locked)	Pass (Fig. 10)
7	Spending a reward that is not mined yet	Both	Rejected: 0 spendable	Pass
8	Reused nonce (0 when 1 is expected)	Account	Rejected: nonce already used	Pass (Fig. 14)
9	Skipped nonce (5 when 1 is expected)	Account	Rejected: nonce skips ahead	Pass (Fig. 14)
10	Replay of a confirmed transfer	UTXO	Rejected: input already spent	Pass (Fig. 12)
11	Replay of a confirmed transfer	Account	Rejected: nonce already used	Pass (Fig. 15)
12	Change output	UTXO	Inputs 9 = 3 payment + 5 change + 1 fee	Pass (Fig. 10)
13	Cloud rental of 5 rounds for only 2 pending blocks	Account	Warning when fees exceed rewards	Pass (Fig. 24)
14	Cloud rental at difficulty 3	UTXO	Unprofitable warning every round, block stays pending	Pass (Fig. 18)
15	--difficulty 7 or menu value 5	-	Rejected (range 1 to 4)	Pass
16	Pool shares	UTXO	Shares follow hash rate; 2% fee; total adds up exactly	Pass (Fig. 11)
17	Switching model between runs	Both	Same balances and total supply	Pass (Fig. 8, 13)
18	Pending reward created in account mode, mined in UTXO mode	Both	Admitted again and confirmed	Pass

19	Reward changed and block re-mined	Both	Signature, transaction and link errors	Pass (Fig. 21)
20	chain.dat / ledger.dat edited on disk	Both	Detected at startup	Pass (Fig. 23)
21	Librarian tries to change the difficulty or tamper	Both	Access denied	Pass
22	Chain file from Formative 1	-	Refused, asks for make clean	Pass

10. Design Choices and Assumptions

- **One block per lending event.** I kept the Formative 1 design of one block per lending record, because the brief asks for each confirmed record to be shown with its own title, member, action, timestamp, reward and signature. One mining run confirms the whole pending pool, mining each block in turn.
- **A treasury pays the rewards.** Rewards are real transfers from a LIBRARY account funded in the genesis block, rather than new coins. This means reward transactions have inputs, change and nonces like any other transaction, so they exercise both models fully.
- **Fee handling.** The brief says the fee is deducted from the token amount before the member is credited. So token_reward stays 10 or 5 on the block, the member receives the reward minus 1, and the fee goes to the miner.
- **Mining rewards.** 50 coins per block plus fees, paid by coinbase transactions to MINER_<username>, to the POOL_M accounts and POOL_OPERATOR, or to the renter and CLOUD_HOST.
- **One log for both models.** Transaction IDs only hash the fields both models share; the UTXO inputs and outputs are worked out by the UTXO model and stored alongside. This lets one ledger.dat be replayed into either model.
- **Integer amounts.** All amounts are stored in hundredths of a coin, so money is never stored as a floating-point number.
- **Atomic saving.** chain.dat, pending.dat and ledger.dat each start with a magic value and the record size, and they are written to a temporary file and then renamed, so a crash cannot leave a half-written file.
- **Assumptions.** The loan period is 14 days. Members have no keys of their own, so the logged-in librarian records and signs transfers for them. The cloud rental fee is paid outside the chain. The pool miners are simulated inside one process.

11. Challenges and Solutions

- **Signing before the chain position is known.** In Formative 1 the signature covered the index and previous_hash. A pending block does not have these yet, so I split the block into signed content (the lending event, reward and tx_id) and a header that the miner fills in. The block hash covers both.

- **A second return was rejected in the UTXO model.** As explained in section 5.1, the treasury's single output was locked by the first pending reward. I let a sender spend its own unconfirmed change.
- **Unfair pool shares.** As explained in section 6.3, I replaced the miner-by-miner loop with attempts processed in time order.
- **Switching models with transactions still pending.** Pending transactions are admitted again under the new session's model when the program starts, and UTXO-only data is cleared when a transaction is confirmed under the account model, so the log can always be replayed under either model.
- **Mining speed.** Hashing the whole header for every attempt was wasteful, so I hash the fixed part once and copy that state for each nonce. Difficulty 4 (about 65,000 attempts) takes around 20 to 40 ms.

12. Conclusion

The extended tracker connects lending, token transactions and mining in one flow. A return creates a reward transaction that is bound to its signed block. The block waits in the pending pool until a miner confirms it with SHA-256 proof of work, and only then does the member's balance change. The same transactions can be managed with the UTXO model (inputs, outputs, change, fees and locked inputs) or the account model (balances, nonces and linked-list history). Solo, pool and cloud mining all confirm the same pending pool and pay their rewards fairly. Validation detects any change to a block, a transaction or the chain links, even after the attacker redoes the proof of work.

Build and run instructions are in README.md.

Appendix: System Design Diagram

The diagram shows the flow from a lending event, through signing and the pending pool, to mining, the confirmed chain and the token ledger with its two models.

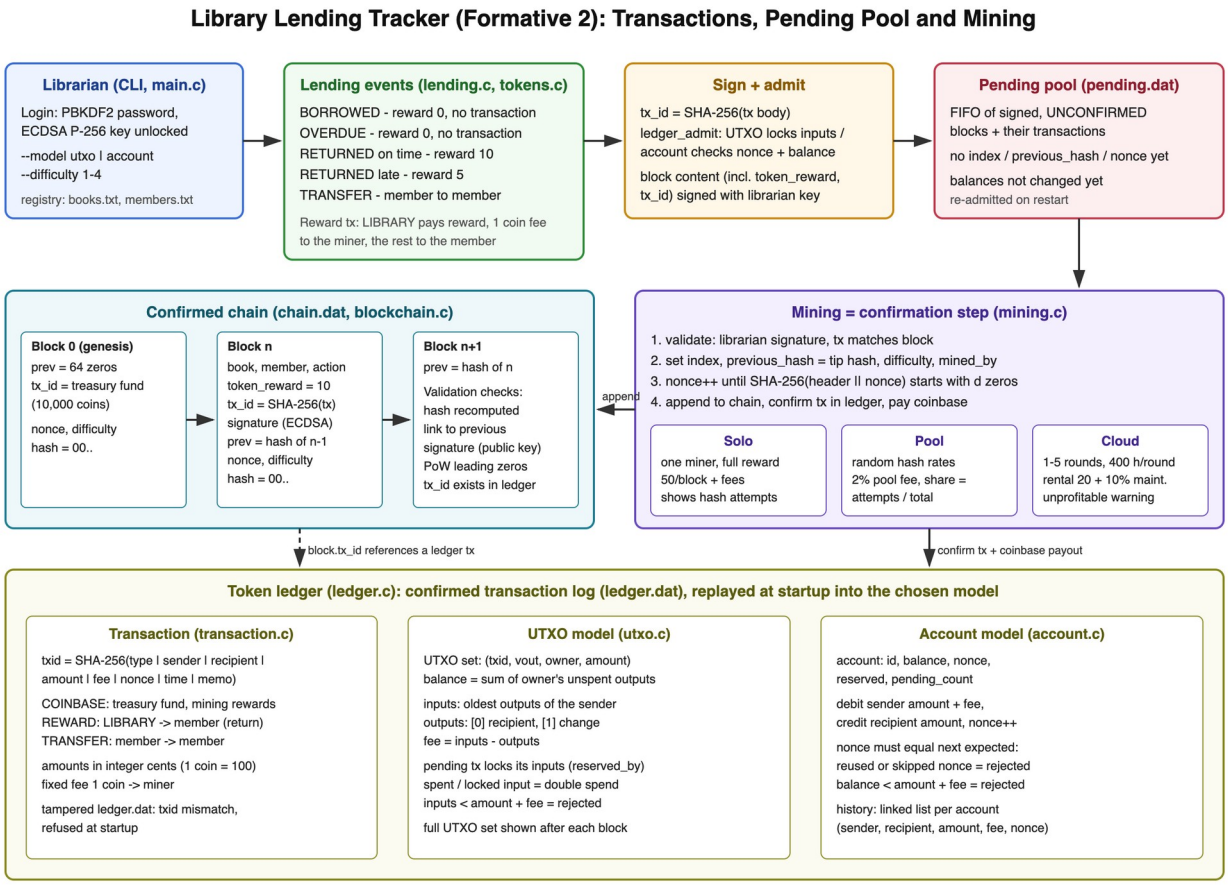


Figure 25: System design: lending events, pending pool, mining, chain and ledger models

Appendix B: Requirements Checklist

Where each requirement of the brief is implemented and where it is shown.

Table 6: Requirements checklist

Requirement	Implementation	Shown in
Return on time: 10-coin reward tx; late: 5; not returned: none	lending_return(), lending_overdue()	Fig. 3, 4
Block waits in pending pool; mining confirms it; then balance updates	app_submit_block(), confirm_front()	Fig. 5, 6, 7
Block fields token_reward and tx_id (SHA-256 of reward tx)	Block in blockchain.h, tx_compute_id()	Table 1, Fig. 9
UTXO: inputs/outputs, UTXO tracking, balance from UTXOs	utxo.c	Fig. 8, 10
UTXO: fee, change output, reject insufficient inputs	build_outputs(), prepare_spend()	Fig. 4, 10
UTXO: prevent double spending	check_inputs(), reserved_by	Fig. 12
UTXO: full set after each confirmed block	show_state_after_block()	Fig. 6, 7
Account: balances, debit/credit,	account_admit(), account_apply()	Fig. 13, 14

insufficient balance rejected		
Account: nonce, wrong/reused nonce rejected	account_admit()	Fig. 14, 15
Account: linked-list history and print command	TxHistoryNode, menu option 12	Fig. 16
Model selection for the whole session	--model, startup menu	Fig. 2, 13
Solo mining, nonce search, hash attempts, full reward	mine_solo(), pow_try()	Fig. 6, 7
Difficulty 1-4, default 2, CLI and menu	--difficulty, mining_set_difficulty()	Fig. 19, 20
Pool: random hash rates, 2% fee, share by attempts, table	mine_pool(), print_pool_table()	Fig. 11
Cloud: 1-5 rounds, rental and maintenance fees, net profit, warning	mine_cloud(), print_cloud_summary()	Fig. 17, 18
Confirmed records with title, member, action, time, reward, signature	chain_print_records()	Fig. 9
Edge cases: no return, insufficient balance, failed nonce, unprofitable cloud	tests/run_tests.sh	Table 5